

Sprout

Autonomous Greenhouse Plant-Care System

A multi-bot mechatronic system for autonomous watering, fertilisation, and plant-health monitoring

ROB-GY 6103: Advanced Mechatronics — NYU Tandon School of Engineering
Group 15 — Devank Thawre, Raj Priyadarshi, Sagarika (Rika) Rao Valluri
11 May 2025

1. Abstract

Greenhouse plants and small gardens grow many different species year-round, each needing customised watering schedules, fertilisation, and continuous soil monitoring. Sprout automates this through a multi-bot architecture driven by real-time sensor data. A 3×2 plant bed is instrumented with capacitive soil-moisture sensors; a mobile **water bot** follows an IR line-following track, stops at each plant via IR object detection, and dispenses a calibrated amount of water through a relay-driven solenoid valve based on that plant's moisture reading. A second **NPK bot** lowers a servo arm to read soil nitrogen, phosphorus, and potassium over an RS-485 Modbus sensor and dispenses compost accordingly. A Raspberry-Pi camera bot serves plant images over a Flask web server with AI plant identification. All inter-device communication uses the ESP-NOW peer-to-peer protocol between ESP32 modules, giving fully wireless, autonomous coordination with no manual intervention.

2. System Architecture

The control stack is distributed across three layers. The **Arduino UNO** reads all analog soil-moisture sensors and stores the values in an indexed array. Three **ESP32 modules** handle path logic and inter-device communication over ESP-NOW (peer-to-peer Wi-Fi that needs no router — devices address one another directly by MAC). A **Raspberry Pi** runs the Flask web server and AI plant-identification feature.

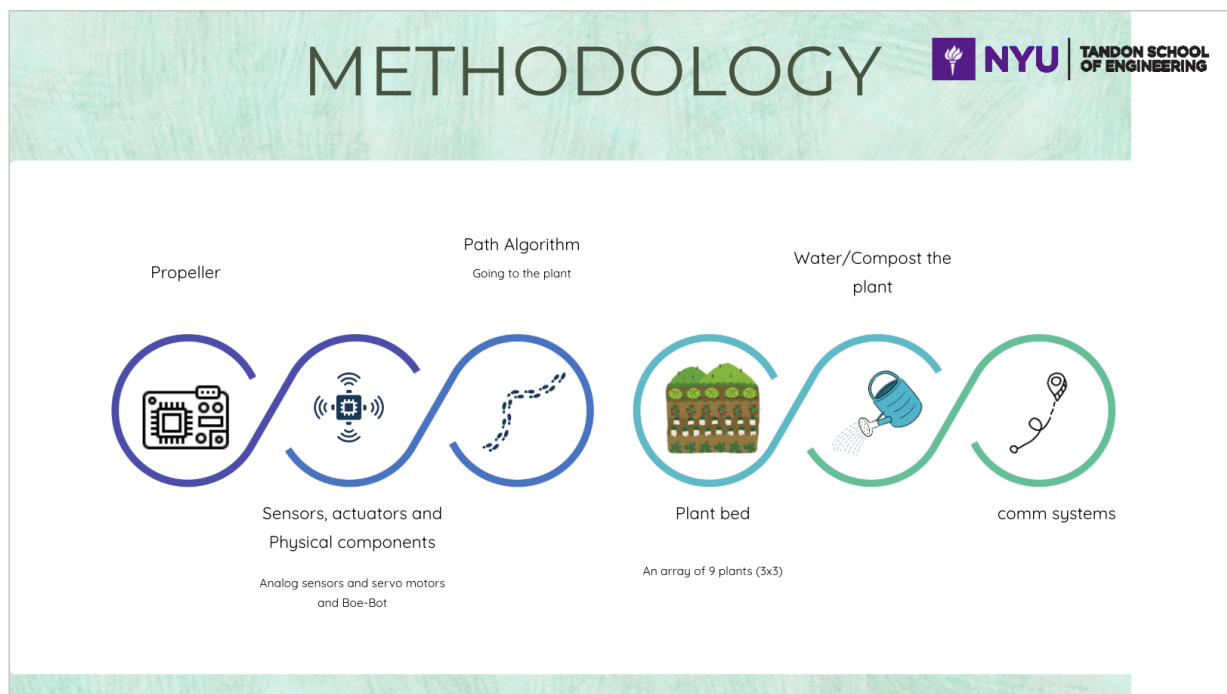


Figure 1 — System methodology: drive → sensors/actuators → path algorithm → 3×2 plant bed → watering/composting → comm systems.

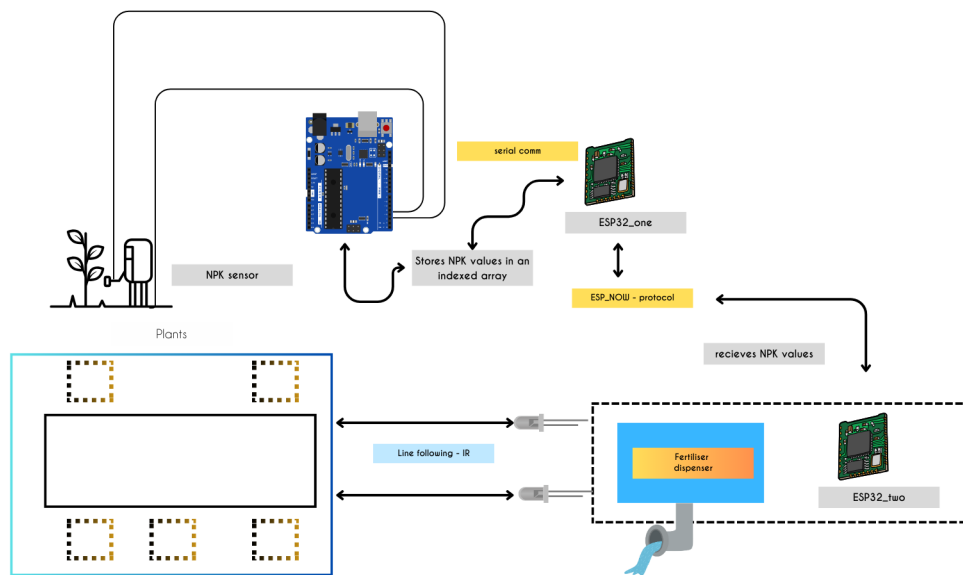


Figure 2 — System block diagram: the NPK sensor feeds the Arduino, which stores readings and forwards them via ESP32_one over ESP-NOW to ESP32_two on the bot, which drives the fertiliser dispenser; IR sensors provide line-following.

3. Circuit & Wiring

The full schematic below shows the two-ESP32 topology around the Arduino UNO. The five capacitive soil sensors connect to the Arduino's analog channels through a 1 kΩ / 2 kΩ resistor network; the ESP32 on the bot drives the relay (NC/NO) that switches the solenoid valve, and the NPK sensor connects over its DATA/VCC/GND lines through the RS-485 converter.

Soil_sensor_5

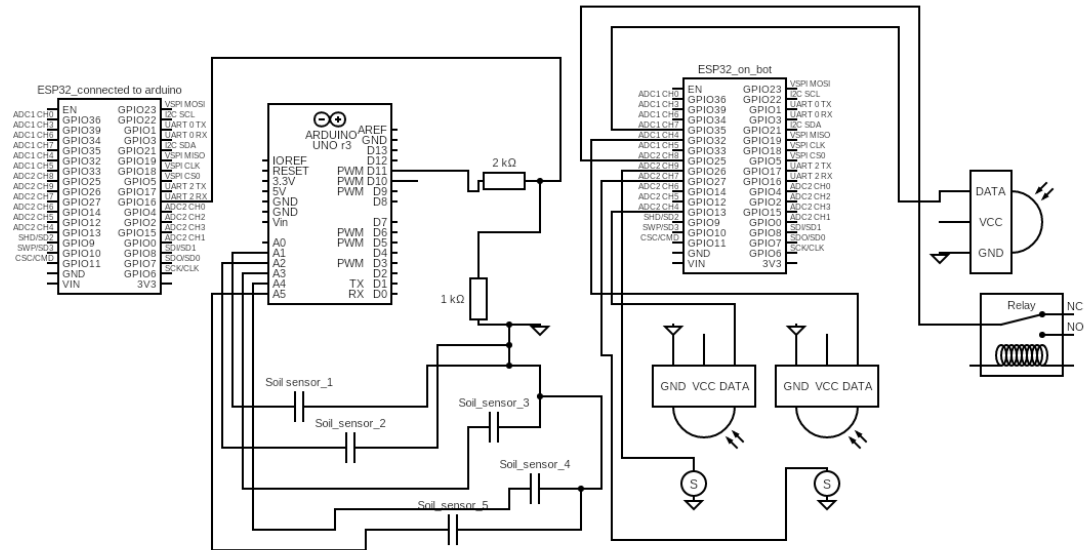


Figure 3 — Full circuit schematic: ESP32 (to Arduino), Arduino UNO R3, ESP32-on-bot, five soil sensors, resistor divider, NPK sensor, relay, and servos.

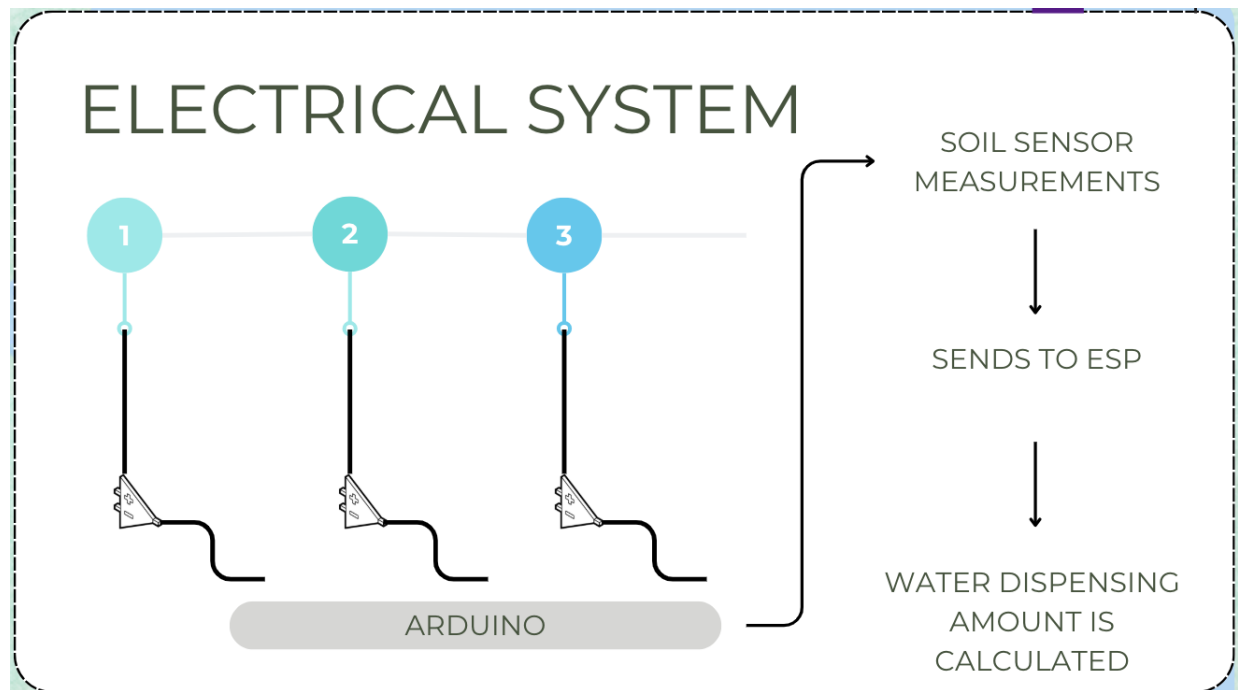


Figure 4 — Electrical-system flow: soil-sensor measurements are sent to the ESP, which calculates the water-dispensing amount.

4. Water Bot — Sensing & Dispensing

Each plant has a capacitive soil-moisture sensor whose output voltage rises as the soil dries (saturated ≈ 1.03 V, dry ≈ 3.3 V). The Arduino converts the raw ADC value with $values[i] = (raw / 1023.0) * 3.3$ and relays it over ESP-NOW to the water bot. The bot follows an IR line, stops at each plant on IR object detection, and actuates the solenoid valve (2 ml per pulse) a number of times set by the moisture reading:

Moisture voltage	Dispense count	Interpretation
≤ 1.03 V	0	Too moist — skip
≤ 1.26 V	0	Still moist — skip
≤ 1.70 V	1×	Slightly dry
≤ 2.66 V	3×	Dry
≤ 2.80 V	4×	Very dry
> 2.80 V	6×	Bone dry

```
#include <esp_now.h>
#include <WiFi.h>
#include <ESP32Servo.h>

// === Sensor & Servo Pins ===
const int leftSensorPin = 25;
const int rightSensorPin = 32;
const int objectSensorPin = 35;

const int leftServoPin = 26;
const int rightServoPin = 27;

#define ARM_SERVO_PIN 14 // for lowering/raising sensor
#define NPK_SERVO_PIN 12 // NEW: for rotating/spinning NPK read arm
const byte nitro[] = {0x01, 0x03, 0x00, 0x1e, 0x00, 0x01, 0xe4, 0x0c};
const byte phos[] = {0x01, 0x03, 0x00, 0x1f, 0x00, 0x01, 0xb5, 0xcc};
const byte pota[] = {0x01, 0x03, 0x00, 0x20, 0x00, 0x01, 0x85, 0xc0};

Servo armServo;
Servo npkServo;

// === State Variables ===
int plantCounter = 0;
bool objectPreviouslyDetected = false;

// === MAC address of ESP1 (receiver) ===
uint8_t esp1MAC[] = {0xCC, 0xDB, 0xA7, 0x92, 0xCC, 0x30};

// === Struct to Send via ESP-NOW ===
typedef struct {
  int id;
  float nitrogen;
  float phosphorus;
  float potassium;
} NPKData;

NPKData npkData;
```

Figure 5 — dispenseWater() thresholds mapping moisture voltage to solenoid pulse count.

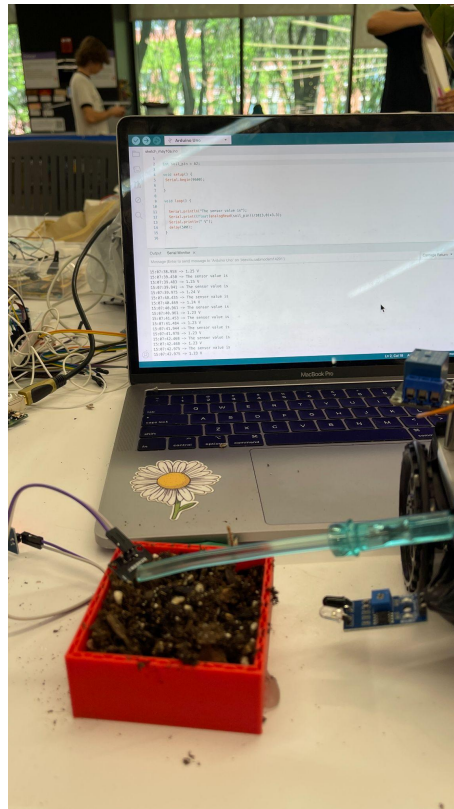


Figure 6 — Two soil profiles characterised by gravimetric/volumetric analysis, each with its own moisture-to-water mapping.

5. NPK (Compost) Bot

A second autonomous bot carries an NPK sensor (RS-485 Modbus RTU) read by an ESP32 through an RE/DE-controlled RS-485 converter. On reaching a plant, a servo arm lowers the probe; the ESP32 writes the Modbus request bytes (e.g. nitrogen = 01 03 00 1e 00 01 e4 0c), toggles the DE/RE direction lines, and reads back the N/P/K values. These are packed into a struct and sent over ESP-NOW (ESP3 → ESP1 → Arduino). Measured readings: Profile 1 N 150 / P 155 / K 145 mg/ml; Profile 2 N 45 / P 70 / K 60 mg/ml. Based on these, the servo-actuated compost flap dispenses fertiliser twice for nutrient-poor soil and once for healthier soil — a closed-loop sensor→actuator response.

```
        delay(3000); // wait before continuing
        driveServos(1300, 1700); // resume
    }
} else if (rightSensor == LOW && leftSensor == HIGH) {
    driveServos(1300, 1500);
} else if (rightSensor == HIGH && leftSensor == LOW) {
    driveServos(1500, 1700);
} else if (rightSensor == HIGH && leftSensor == HIGH) {
    driveServos(1500, 1700);
} else {
    driveServos(1500, 1500);
}

if (objSensor == HIGH && objectPreviouslyDetected) {
    objectPreviouslyDetected = false;
}

delay(50);
}

void measureAndSendNPK() {
    Serial.println("📡 Lowering sensor...");
    armServo.write(100);
    delay(1000); // arm stays down

    byte nitrogen() {
        digitalWrite(DE, HIGH);
        digitalWrite(RE, HIGH);
        delay(10);

        if (mod.write(nitro, sizeof(nitro)) == 8) {
            digitalWrite(DE, LOW);
            digitalWrite(RE, LOW);
            delay(10);

            for (byte i = 0; i < 7; i++) {
                values[i] = mod.read();
                Serial.print(values[i], HEX);
                Serial.print(" ");
            }
        }
    }
}
```

Figure 7 — Pin map, servo definitions, and the raw NPK Modbus request byte-arrays.


```

#include <esp_now.h>
#include <WiFi.h>

#define RXD2 16
#define TXD2 17

typedef struct {
    int id;
    float nitrogen;
    float phosphorus;
    float potassium;
} NPKData;

NPKData npk;

void setup() {
    Serial.begin(115200);
    Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2); // TX to Arduino

    WiFi.mode(WIFI_STA);

    if (esp_now_init() != ESP_OK) {
        Serial.println("❌ ESP-NOW init failed");
        return;
    }

    esp_now_register_recv_cb(OnDataRecv);
    Serial.println("✅ ESP1 ready to receive NPK from ESP3 and forward to Arduino");
}

void OnDataRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
    if (len == sizeof(npk)) {
        memcpy(&npk, data, sizeof(npk));

        Serial.println("📁 Received NPK:");
        Serial.print("Plant "); Serial.println(npk.id);
        Serial.print("N: "); Serial.println(npk.nitrogen);
        Serial.print("P: "); Serial.println(npk.phosphorus);
        Serial.print("K: "); Serial.println(npk.potassium);
    }
}

```

Figure 8 — ESP-NOW forwarder: ESP1 receives the NPK struct from ESP3 and forwards it to the Arduino over Serial2.

6. Hardware & Testing



Figure 9 — The mobile bot rig: drive wheels, relay-switched solenoid, IR sensor, and the instrumented soil pot.

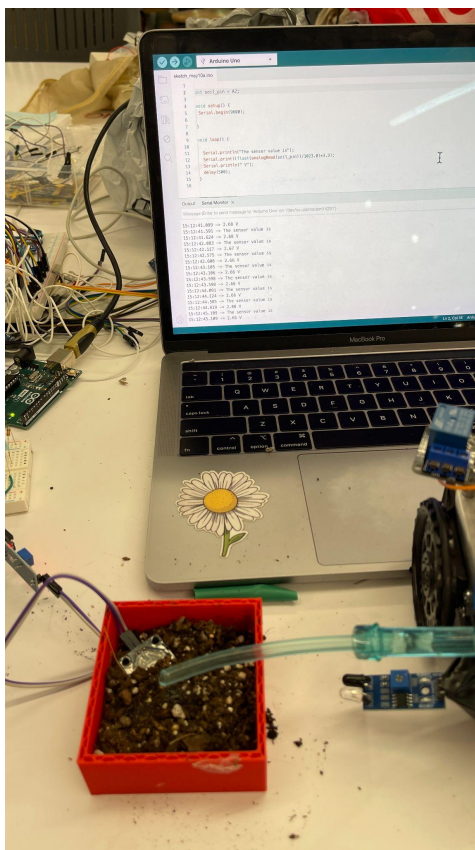


Figure 10 — Capacitive sensor in a pot with the Arduino serial monitor showing live voltage readings during calibration.

7. Results, Cost & Lessons

Results: soil-moisture voltages mapped cleanly to dispense counts; NPK readings were repeatable across two soil profiles; and autonomous multi-bot coordination was demonstrated on a physical track with no manual intervention. **Cost:** ~\$136.72 using class-kit components (Arduino, 6 soil sensors, 3 ESP32s, Raspberry Pi); ~\$387 costed with individual components; a mass-production estimate reaches ~\$3,550 for 50 units (excluding Boe-Bot).

Lessons: gained hands-on experience with UART/serial and Wi-Fi (ESP-NOW) protocols, integrated RS-485 Modbus with the ESP32 for the NPK sensor, built soil profiles via gravimetric water-content analysis to calibrate the watering logic, used Flask to bridge hardware and a web interface, and engineered robust multi-bot path-planning and plant-counter synchronisation across independent systems.